

Experiment -1

Linux Programming Environment & Basic Commands

Aim

To set up a Linux programming environment, compile and execute a simple C program, and study commonly used Linux terminal, file-handling, permission, and redirection commands.

Learning Objectives

- Identify online and offline methods for obtaining a Linux programming environment.
- Differentiate Type-1 and Type-2 hypervisors and install a Linux virtual machine using Oracle VirtualBox.
- Navigate the Linux file system and create, inspect, copy, move, remove, and protect files.
- Use output, append, and input redirection in shell commands.

Requirements

A 64-bit computer with hardware virtualization enabled, approximately 4 GB RAM available for the guest, 30 GB free storage, Oracle VirtualBox, a pre-built Kali Linux VirtualBox image, and an internet connection for downloads.

1. Setting up of Linux Programming Environment

C programming can be performed online or offline. In both cases a Linux shell, a text editor, a C compiler such as GCC, and permission to create and execute files are required.

Online Method: CoCalc

CoCalc provides a browser-based Linux terminal and project storage. A typical procedure is:

- Open <https://cocalc.com>, create an account or sign in, and create a new project.
- Open the project, select New, and start a Linux Terminal.
- Create a source file using vi hello.c, save it, compile with gcc hello.c -o hello, and run it with ./hello.
- Save the source file and terminal output in the project. This method needs a stable internet connection and the features available to the selected account.

Offline Methods

- Dual boot: Linux and the existing operating system are installed on separate partitions. The user selects one at startup. It gives Linux direct hardware performance but partitioning and boot-loader changes require care.
- Single boot: Linux is the only operating system on the computer. It offers full resource availability but normally removes or replaces the earlier operating system and its applications.
- Bootable pendrive (Live USB): Linux starts from a USB drive without a permanent installation. It is useful for practice, testing, and recovery; changes are temporary unless persistent storage is configured.
- Virtual machine: Linux runs as a guest inside the current host operating system. It is isolated, easy to reset, and convenient for laboratories, although the host and guest share CPU, memory, and storage.

2. Virtual Machine Manager (Hypervisor)

A hypervisor is software or firmware that creates and manages virtual machines by allocating physical CPU, memory, storage, and network resources to isolated guest operating systems.

Type-1 (Bare-metal) Hypervisor

A Type-1 hypervisor runs directly on the computer hardware. It is generally used in servers and data centres because it provides strong isolation, central management, and low virtualization overhead. Examples include VMware ESXi, Microsoft Hyper-V, Xen, and KVM-based enterprise platforms.

Type-2 (Hosted) Hypervisor

A Type-2 hypervisor runs as an application on a host operating system such as Windows, macOS, or Linux. It is simple to install and is well suited to classrooms and personal computers. Examples include Oracle VirtualBox, VMware Workstation, and Parallels Desktop. Oracle VirtualBox is used in this experiment.

3. Installation of VMM & VM

Step-1: Download Oracle VirtualBox for Windows hosts.

Open the VirtualBox Downloads page and select Windows hosts. Complete the download from the official source.

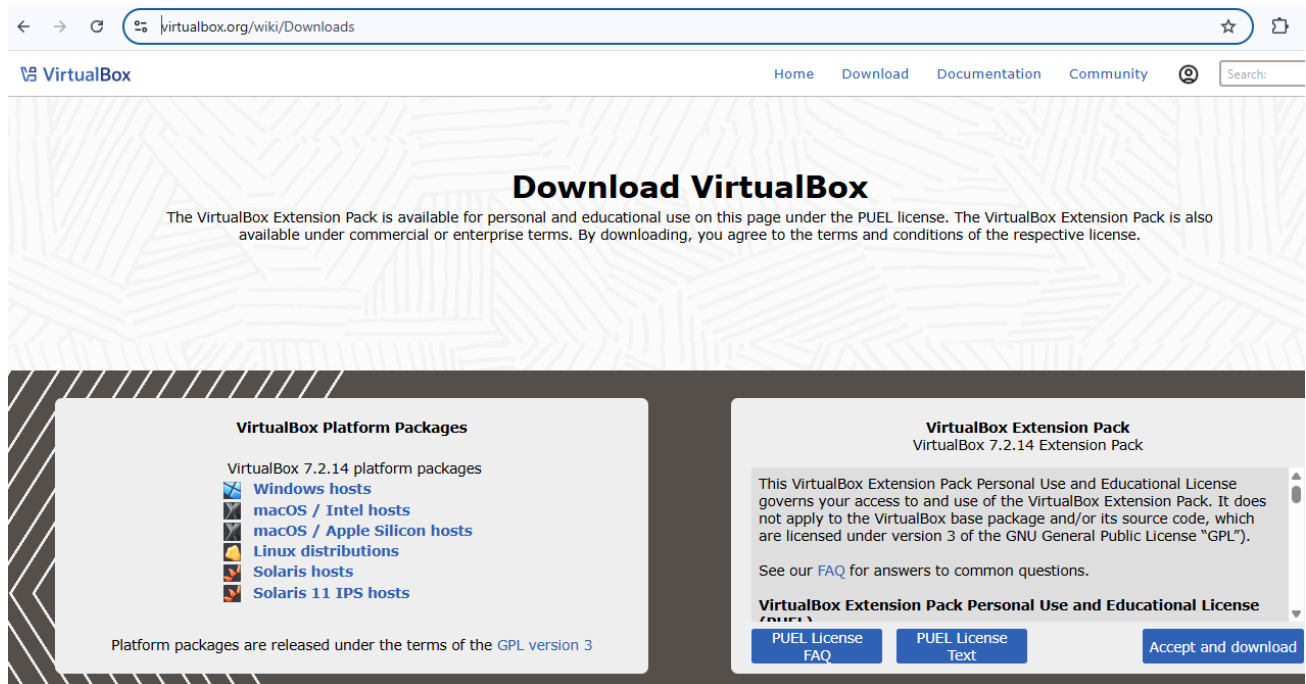


Figure 1: Oracle VirtualBox download page for selecting the Windows host package.

Step-2: Download the pre-built Kali Linux virtual machine for VirtualBox.

Open the Kali Linux Virtual Machines download page, select the 64-bit VirtualBox image, and download the archive. A pre-built image avoids a full operating-system installation inside the VM.

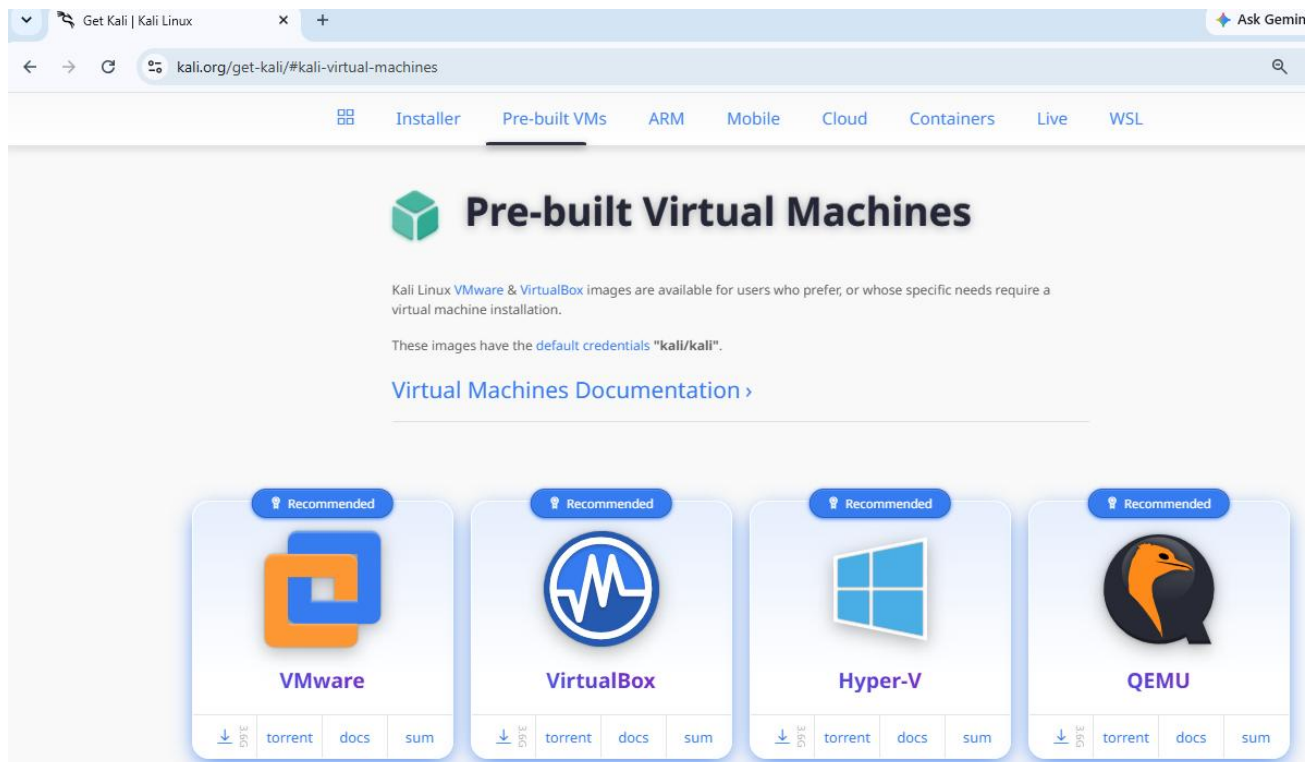


Figure 2: Kali Linux pre-built virtual machine download option.

Step-3: Extract the downloaded Kali Linux archive.

Use a suitable archive utility and extract the files to a folder with sufficient free space. Keep the virtual disk and machine configuration files together.

Step-4: Install Oracle VirtualBox.

Run the downloaded installer, accept the required network and device-driver prompts, keep the required features selected, and complete the installation.

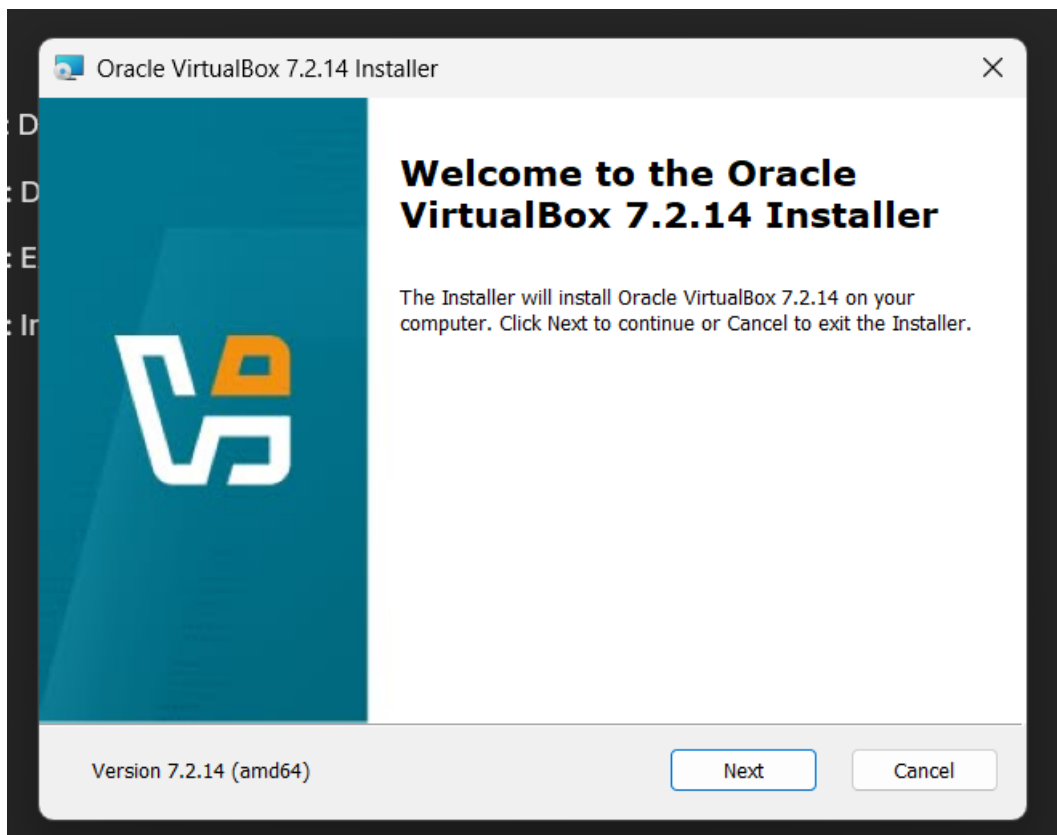


Figure 3: Oracle VirtualBox installation wizard.

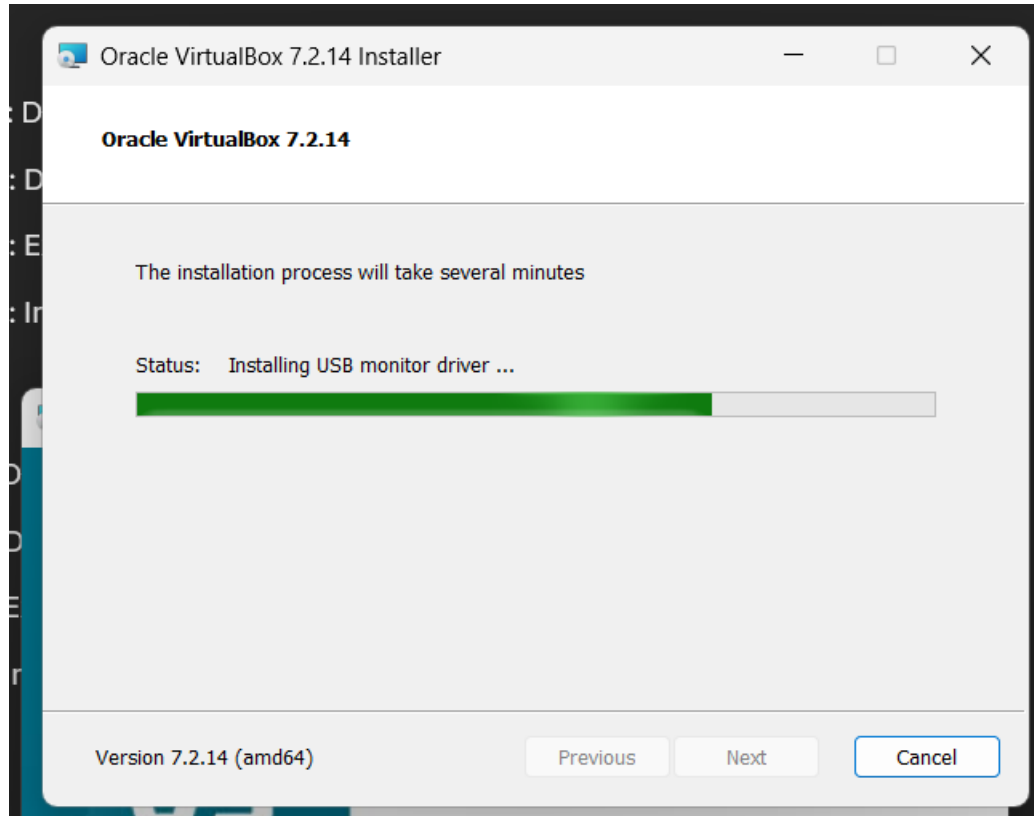


Figure 4: Completing the VirtualBox installation.

Step-5: Open Oracle VirtualBox.

Start VirtualBox from the Start menu. The VirtualBox Manager window displays available virtual machines and their current state.

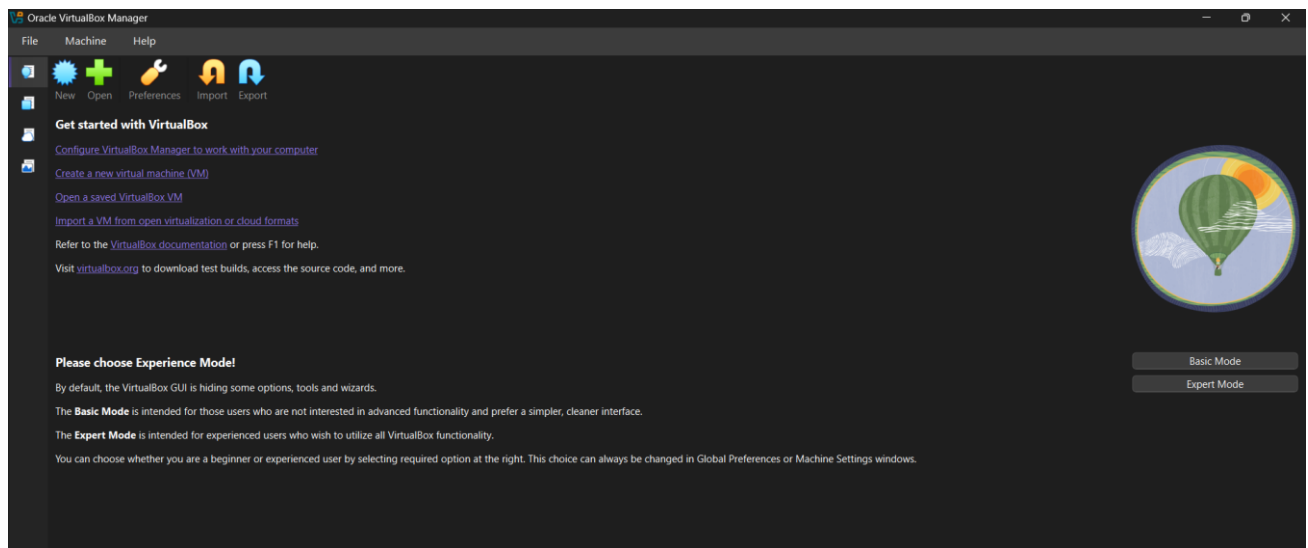


Figure 5: Oracle VirtualBox Manager.

Step-6: Open the saved Kali Linux virtual machine.

Choose Machine > Add (or open the extracted .vbox file), select the saved Kali Linux configuration, and verify the VM settings. If necessary, assign approximately 2 processor cores and 4 GB memory without exceeding the host computer limits.

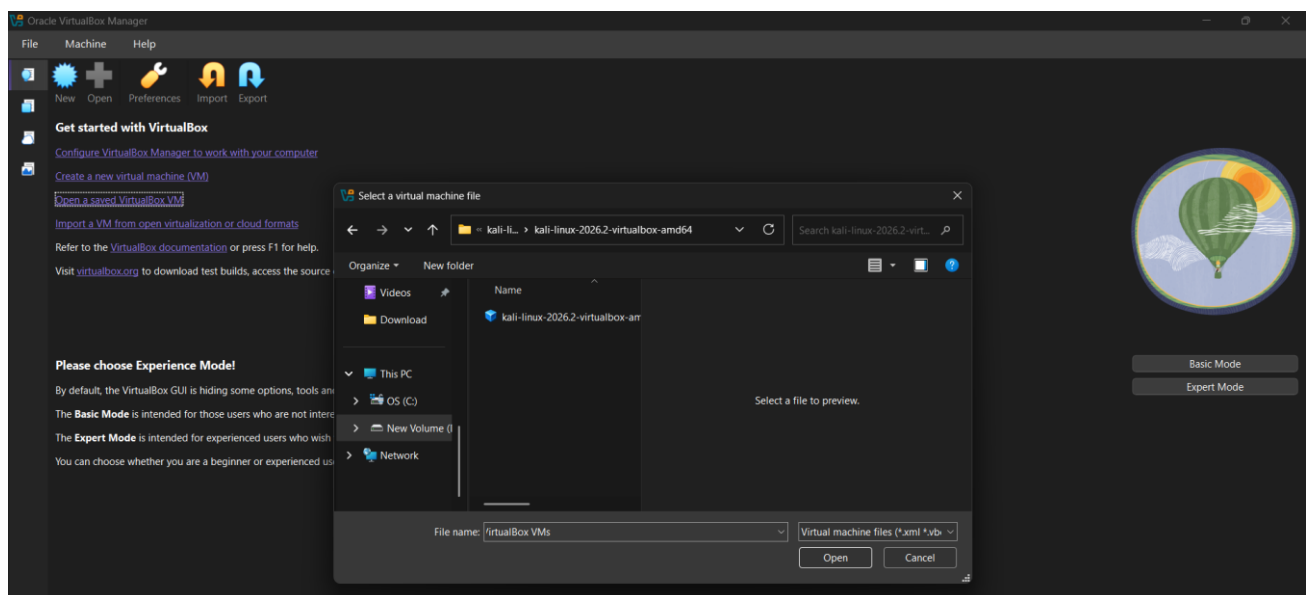


Figure 6: Opening the extracted Kali Linux virtual machine.

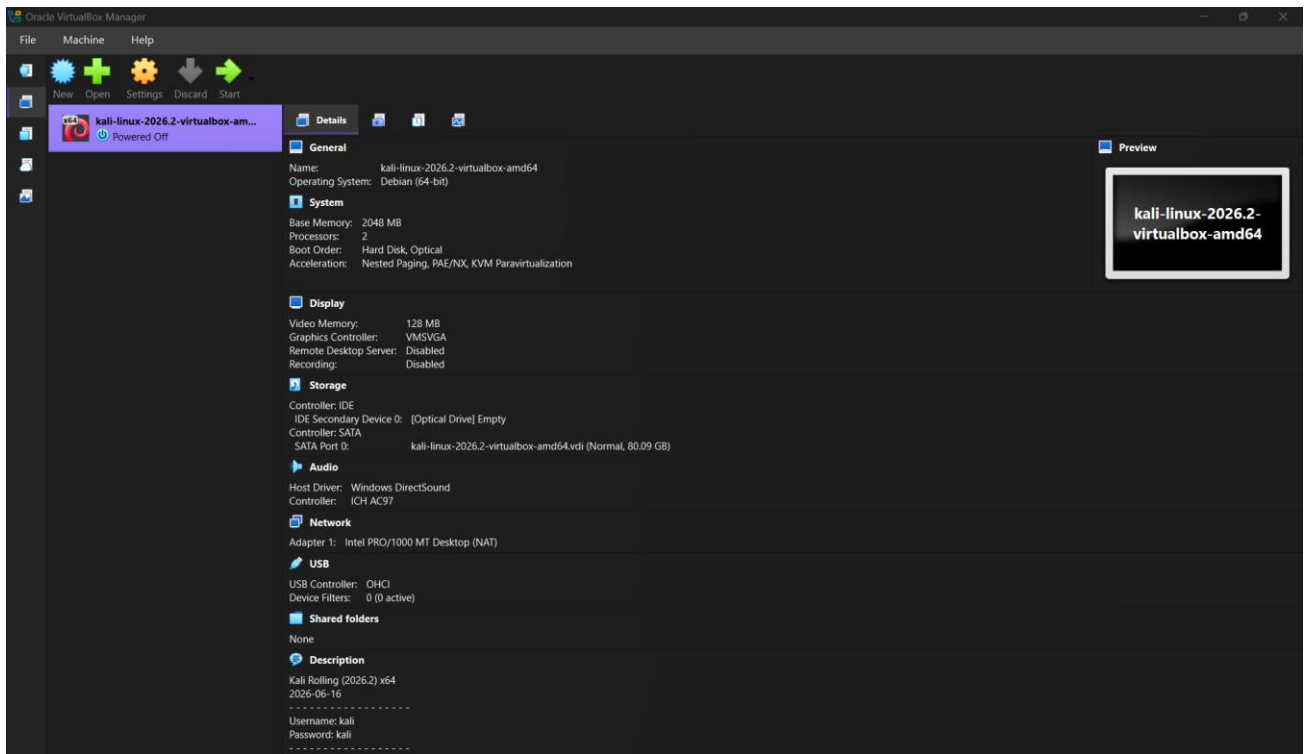


Figure 7: Kali Linux virtual machine registered in VirtualBox.

Step-7: Start the virtual machine.

Select the Kali Linux VM and choose Start. After the boot process, log in with the credentials supplied for the downloaded image. The guest desktop confirms that the Linux environment is ready.

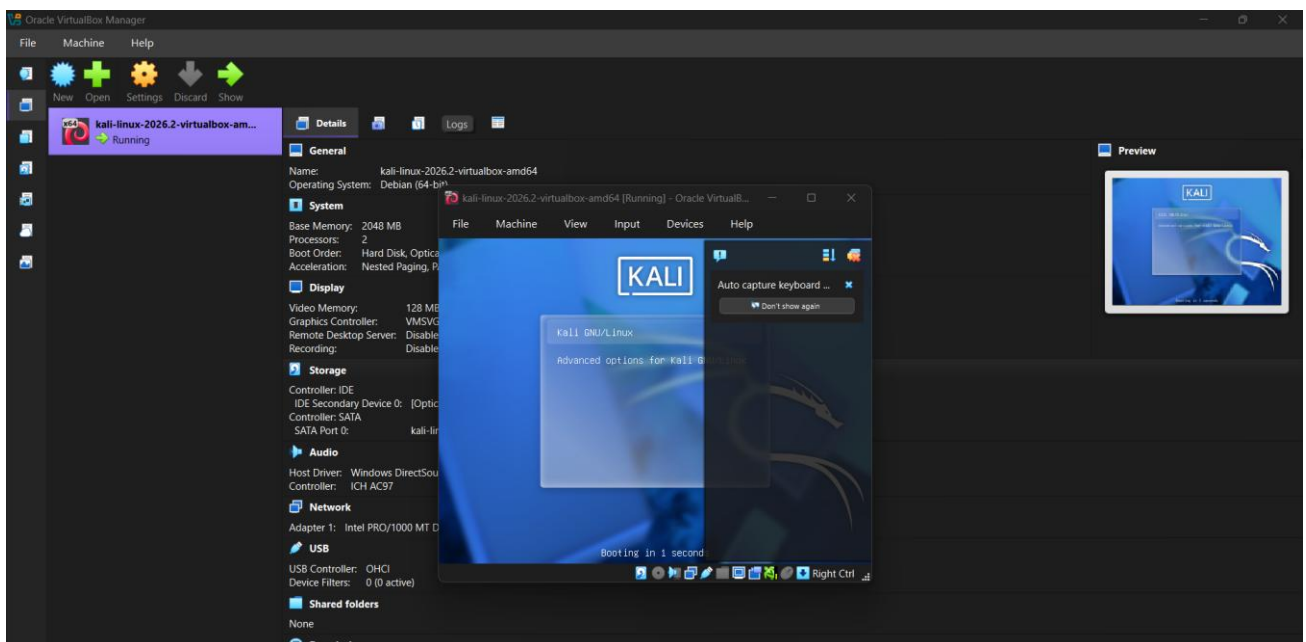


Figure 8: Starting the Kali Linux guest in VirtualBox.

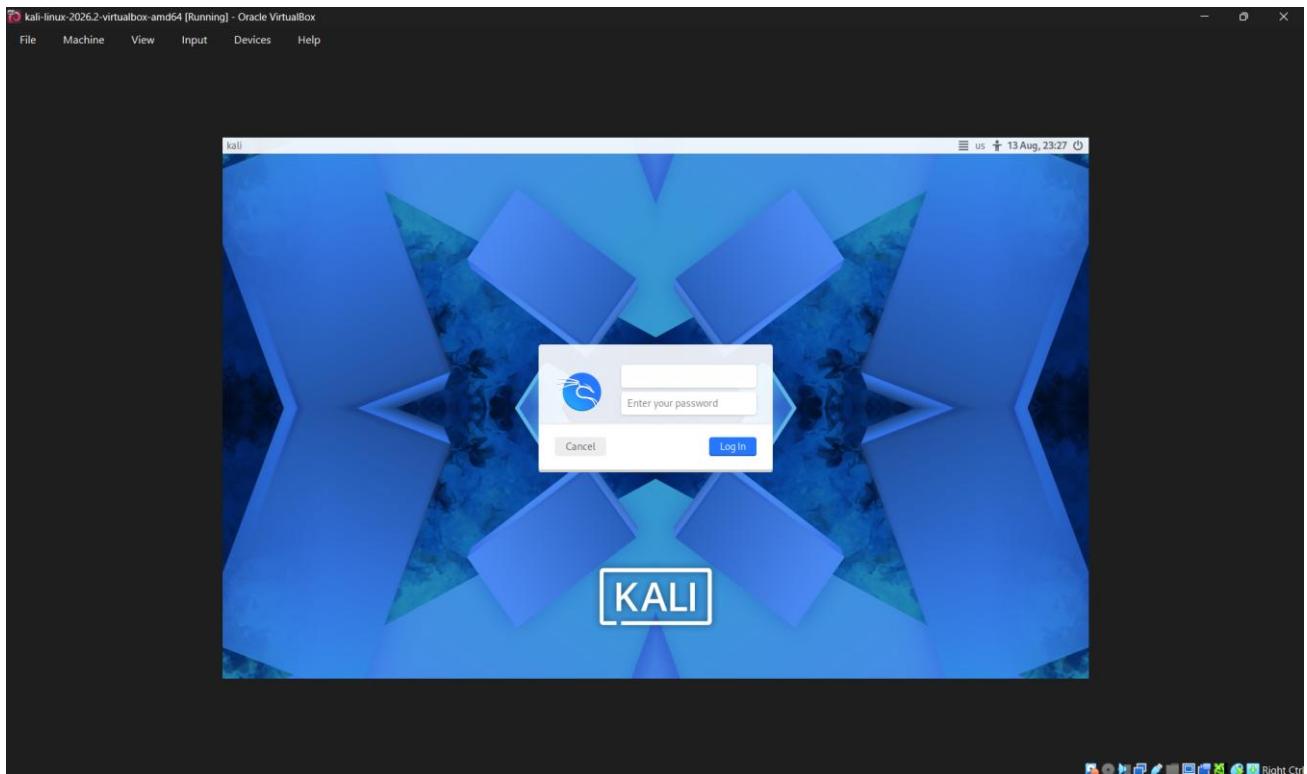


Figure 9: Kali Linux login screen running inside Oracle VirtualBox.

Thus, the Type-2 hypervisor (Oracle VirtualBox) and the guest virtual machine (Kali Linux) are available for Linux programming practice.

Step-8: Create, compile, and execute a simple C program.

Open a terminal, create hello.c with vi, save the program, compile it with GCC, and execute the generated program. GCC normally prints no message when compilation succeeds.

```
#include <stdio.h>
int main(void) {
    printf("Linux programming environment is ready!\n");
    return 0;
}
```

```
C program: create, compile and execute

user@kali:~$ mkdir -p linux_lab && cd linux_lab
user@kali:~/linux_lab$ vi hello.c
# In vi: press i, enter the program, press Esc, type :wq, then Enter
user@kali:~/linux_lab$ cat hello.c
#include <stdio.h>
int main(void) {
    printf("Linux programming environment is ready!\n");
    return 0;
}
user@kali:~/linux_lab$ gcc hello.c -o hello
user@kali:~/linux_lab$ ./hello
Linux programming environment is ready!
```

Figure 10: Creating, compiling, and executing a C program in the Linux terminal.

4. Working with Basic Linux Commands

The following commands were practiced in a working directory named ~/linux_lab. Linux commands and file names are case-sensitive. The prompt shown in the sample outputs is not typed as part of a command.

4.1 Terminal Basics

Command	Purpose	Example
pwd	Print the absolute path of the present working directory.	pwd
mkdir	Create one or more directories. Option -p also creates missing parent directories.	mkdir docs temp
rmdir	Remove an empty directory. It fails if the directory contains files.	rmdir temp
dir	List directory contents using the GNU dir program.	dir
ls	List files and directories. -l gives details and -a includes hidden entries.	ls -la
cd	Change the current directory; cd with no argument returns to the home directory.	cd docs
cd ..	Move to the parent of the current directory.	cd ..
clear	Clear the visible terminal screen without deleting command history.	clear
history	Display commands previously entered in the current shell history.	history tail
man	Open the manual page for a command. Press q to exit.	man ls
whoami	Print the effective user name of the current user.	whoami

```
Terminal basics: location, directories and listing

user@kali:~$ pwd
/home/user
user@kali:~$ mkdir linux_lab
user@kali:~$ cd linux_lab
user@kali:~/linux_lab$ mkdir docs temp
user@kali:~/linux_lab$ ls
docs  temp
user@kali:~/linux_lab$ dir
docs  temp
user@kali:~/linux_lab$ cd docs
user@kali:~/linux_lab/docs$ cd ..
user@kali:~/linux_lab$ rmdir temp
user@kali:~/linux_lab$ ls -la
drwxr-xr-x 3 user user 4096 Aug 19 10:25 .
drwx----- 8 user user 4096 Aug 19 10:20 ..
drwxr-xr-x 2 user user 4096 Aug 19 10:24 docs
```

Figure 11: Working-directory, directory-creation, listing, navigation, and removal commands.

```
Terminal basics: identity, help, history and screen clearing

user@kali:~/linux_lab$ whoami
user
user@kali:~/linux_lab$ history | tail -n 5
16 mkdir docs temp
17 ls
18 dir
19 rmdir temp
20 history | tail -n 5
user@kali:~/linux_lab$ man ls
LS(1)                                User Commands                                LS(1)
NAME
    ls - list directory contents
Press q to leave the manual page.
user@kali:~/linux_lab$ clear
The terminal display is cleared; the command history remains available.
```

Figure 12: Identity, history, manual-page, and clear commands.

Observations

- pwd changes after cd enters another directory; cd .. returns to the parent directory.
- Both ls and dir list entries, while ls is more commonly used and provides options such as -l, -a, and -h.
- clear affects only the display. Earlier commands remain accessible through history and the shell history file.

4.2 File Commands

Command	Purpose	Example
vi	Open the vi editor. Press i to insert, Esc to leave insert mode, and :wq to save and quit.	vi notes.txt
file	Inspect file data and report the likely file type.	file notes.txt
rm	Remove files. Deletion is normally immediate, so verify the target first.	rm old.txt
cp	Copy a file or directory. Use -r when copying a directory tree.	cp notes.txt backup/
mv	Move a file to another location or rename it.	mv old.txt new.txt
cat	Display or combine complete text files.	cat notes.txt
more	View long text one screen at a time, mainly moving forward.	more numbers.txt
less	View and search long text with forward and backward movement.	less numbers.txt
head	Display the first ten lines by default, or a specified number with -n.	head -n 2 notes.txt
tail	Display the last ten lines by default; -f can follow a growing file.	tail -n 1 notes.txt
chmod	Change permission bits using symbolic or octal notation.	chmod 640 notes.txt

```
Files: vi, cat, file, head and tail

user@kali:~/linux_lab$ vi notes.txt
# In vi: i -> type text -> Esc -> :wq -> Enter
user@kali:~/linux_lab$ cat notes.txt
Linux is multiuser and multitasking.
The shell reads commands and starts programs.
Permissions protect files and directories.
user@kali:~/linux_lab$ file notes.txt
notes.txt: ASCII text
user@kali:~/linux_lab$ head -n 2 notes.txt
Linux is multiuser and multitasking.
The shell reads commands and starts programs.
user@kali:~/linux_lab$ tail -n 1 notes.txt
Permissions protect files and directories.
```

Figure 13: Editing and inspecting a text file with vi, cat, file, head, and tail.

```
Files: cp, mv, chmod and rm

user@kali:~/linux_lab$ mkdir backup
user@kali:~/linux_lab$ cp notes.txt backup/notes-copy.txt
user@kali:~/linux_lab$ ls backup
notes-copy.txt
user@kali:~/linux_lab$ mv backup/notes-copy.txt backup/linux-notes.txt
user@kali:~/linux_lab$ chmod 640 backup/linux-notes.txt
user@kali:~/linux_lab$ ls -l backup/linux-notes.txt
-rw-r----- 1 user user 142 Aug 19 10:42 backup/linux-notes.txt
user@kali:~/linux_lab$ rm backup/linux-notes.txt
user@kali:~/linux_lab$ rmdir backup
user@kali:~/linux_lab$ ls
docs  notes.txt
```

Figure 14: Copying, renaming, changing permissions, and removing files.

```
Files: more and less pagers

user@kali:~/linux_lab$ seq 1 40 > numbers.txt
user@kali:~/linux_lab$ more numbers.txt
1
2
3
...
--More-- (20%)
Space: next page | Enter: next line | q: quit
user@kali:~/linux_lab$ less numbers.txt
1
2
3
numbers.txt (END)
Arrow keys: move | /text: search | q: quit
```

Figure 15: Viewing a long file with the more and less pagers.

Understanding chmod 640

The three octal digits represent permissions for the owner, group, and others. Read = 4, write = 2, and execute = 1. Therefore 640 gives the owner read + write (6), the group read only (4), and others no permission (0). Symbolic notation can express the same change, for example `chmod u=rw,g=r,o= notes.txt`.

4.3 Redirection

Command	Purpose	Example
>	Send standard output to a file, creating it or replacing its previous contents.	echo "First" > report.txt

Command	Purpose	Example
>>	Append standard output to the end of a file without replacing existing contents.	echo "Second" >> report.txt
<	Use a file as standard input for a command.	wc -l < report.txt

```

Redirection: overwrite, append and standard input

user@kali:~/linux_lab$ echo "First line" > report.txt
user@kali:~/linux_lab$ echo "Second line" >> report.txt
user@kali:~/linux_lab$ cat report.txt
First line
Second line
user@kali:~/linux_lab$ wc -l < report.txt
2
user@kali:~/linux_lab$ sort < report.txt > sorted.txt
user@kali:~/linux_lab$ cat sorted.txt
First line
Second line
> creates/replaces output; >> appends; < supplies a file as input.

```

Figure 16: Output, append, and input redirection.

Precautions

- Use official download sources and verify that enough disk space and memory are available before creating a virtual machine.
- Do not allocate all host RAM or processor cores to the guest VM; the host operating system must remain responsive.
- Check the current path with `pwd` and inspect targets with `ls` before using `rm`, `mv`, or output redirection with `>`.
- Remember that `>` replaces existing file contents, while `>>` preserves them and appends new output.
- Avoid overly broad write or execute permissions; grant only the access actually required.

Result

The Linux programming environment was set up using Oracle VirtualBox and a Kali Linux virtual machine. A C program was created, compiled, and executed successfully. The required terminal, file-handling, permission, paging, and redirection commands were studied with representative outputs.